

SecureLock

Presentation guide — SIH internal hackathon, round 1

EVERYTHING YOU NEED TO PRESENT

We don't just secure the question paper. We secure the question.

Problem, existing systems, novelty, stack, demo, limitations,
and the questions judges actually ask.

This guide is written in plain language so any member of the team can present with it, including someone who did not write the code. Read sections 1 to 5 if you have ten minutes. Read all of it if you are the one answering questions.

1. The first thirty seconds

Open with the problem, not the technology. Judges hear “blockchain” forty times a day; they hear a sharp problem statement rarely.

Say this, roughly

“When an exam paper leaks, everyone pictures the sealed envelope being opened early. But most leaks don't happen there. They happen weeks earlier, while the questions are still being written and emailed between professors — before a ‘paper’ even exists to guard.

SecureLock secures the whole life of a question, from the moment somebody types it to the moment the exam begins. And the final paper is held shut by a rule on a blockchain that decides the opening moment using its own clock — not our server's, and not yours. I can prove that in about thirty seconds.”

Then go straight to the clock demonstration. Do not save it for the end; it is the thing they will remember, and it is better shown early while attention is highest.

2. The problem statement

A question paper is not created as a single document. It is assembled over weeks, by many people:

- 1 A professor writes a question on a personal laptop.
- 2 She emails it to a colleague for a second opinion.
- 3 He pastes it into a shared document with his own suggestions.
- 4 Somebody prints a draft to read on the way home.
- 5 A coordinator collects thirty questions into one file and circulates it for approval.
- 6 Only now does a “question paper” exist — and only now do most security systems start protecting it.

By that point the content has lived on many machines, in several inboxes, and in at least one printout nobody can account for. Every one of those steps is a leak opportunity, and none of them is recorded anywhere that survives a determined insider.

There is a second problem underneath the first. Even a perfectly guarded paper has to be *released* at some moment, and something has to decide when that moment has arrived. Whatever makes that decision becomes the weakest point in the system.

The problem in one line

Protect every question from the moment it is written — and make the release decision something no single person can move.

3. How papers are protected today, and where each protection breaks

These are the approaches currently in use across boards and universities. Each is sensible. Each has a specific hole, and the holes are all the same shape: they depend on trusting whoever holds the most privilege.

What is used today	How it works	Where it breaks
Sealed physical packets	Papers printed at a confidential press, sealed, moved under escort, stored in a strongroom, opened at a fixed hour before witnesses.	Protects the printed paper only. The questions existed digitally for weeks beforehand, entirely outside this protection.
Confidential printing presses	A small trusted vendor prints under supervision.	Concentrates trust in one organisation, and again begins after the questions are already written.
Encrypted file with a password released on exam day	Centres receive an encrypted paper in advance; the password is sent on the morning of the exam.	The encrypted file is already sitting at every centre. Security rests entirely on one password held by a small group — and on them not sending it early.
Server-side timed release	A central server refuses to hand over the paper until a set time.	The server's clock is set by whoever administers the server. An insider with that access can bring the release forward, and the logs that would show it are on the same server.
ERP exam modules with roles and audit logs	Role-based access, with a database log of who did what.	The log is a database table. A database administrator can edit it. A record that its own keeper can quietly rewrite is not evidence.

Notice the pattern. Every one of these protects the *final artefact*, and every one leaves at least one person who can quietly override it — usually the person best placed to do so without being noticed.

4. What we built

SecureLock is a working prototype, not a slide deck. It runs offline on one laptop and does all of the following, verified by automated tests:

- Encrypts **every question separately**, each with its own key, at the moment it is written. The readable text is never stored.
- Gives every question a **fingerprint** (SHA-256) and records that fingerprint on a blockchain, along with an anonymous marker for who wrote it and when.
- Treats **READ, WRITE and APPROVE as three separate permissions**, checked on the server for every request — a reviewer can approve without ever being able to edit.

- Assembles the paper automatically from the approved pool, balancing the blueprint, avoiding near-duplicates and spreading authorship, so **no setter can predict which of their questions was used**.
- Seals the finished paper and registers it with a release time in a **smart contract**, which then refuses to permit release until the blockchain's own clock passes that time.
- Records every action — including every refusal — in a **hash-chained audit log** where editing one row breaks every row after it, visibly.

5. What is genuinely new

Be honest here. AES, SHA-256 and smart contracts are ordinary, well-known technology. Claiming to have invented them would be caught instantly. The contribution is **where** they are applied and **what is deliberately left out**.

Novelty	Why it matters
The question is the unit of security, not the paper	Everything else on the market starts protecting at the point a paper exists. We start at the keystroke. This is the core idea and the whole reason the system is shaped the way it is.
Per-question keys, not one paper key	There is never a moment when one secret unlocks everything. Compromising one key costs one question.
The release condition is not ours to move	The decision lives in a smart contract and reads <code>block.timestamp</code> . Not our server clock, not the user's device. No administrator on our side can bring an exam paper forward.
Refusals are first-class evidence	Most systems log successes. We record every denial and anchor it. An investigation usually needs the attempts that failed, not the ones that worked.
Blockchain used in exactly two places, and argued for	Only the provenance anchor and the release condition. Everything else is an ordinary database, because for those jobs a database is the correct tool. Being able to say where blockchain is <i>not</i> used is itself unusual.
It fails honestly	If the chain is unreachable, the system refuses release rather than falling back to a server clock, and records the attempt instead of inventing a transaction hash that would look convincing and mean nothing.

6. Technology stack

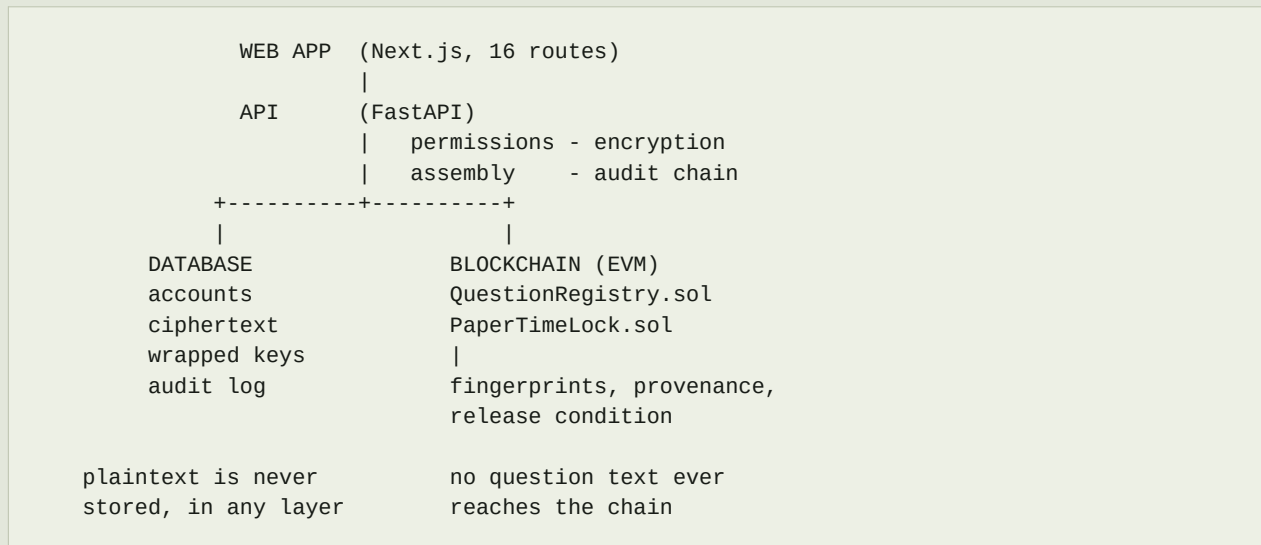
Every choice below is boring on purpose. Nothing here is experimental, because an examination system is not the place for novelty in the plumbing.

Layer	What we used	Why
Smart contracts	Solidity 0.8.24, OpenZeppelin AccessControl 5.1, Hardhat 2.22	Two contracts: QuestionRegistry (provenance) and PaperTimeLock (release). OpenZeppelin because writing your own access control is how contracts get broken.
Blockchain	Local EVM chain (Hardhat), chain ID 31337	Runs offline, needs no internet, no faucet and no real currency. Any EVM-compatible network works by changing one setting.
Backend	Python 3.11-3.13, FastAPI 0.115, SQLAlchemy 2.0, Pydantic 2.10	FastAPI gives typed request validation and interactive API docs for free, which matters when judges ask to see the real endpoint.

Layer	What we used	Why
Cryptography	AES-256-GCM and SHA-256 via cryptography 44; Argon2id for passwords; JWT for sessions	Standard primitives from a maintained library. We never hand-roll cryptography. GCM also detects tampering with the ciphertext itself.
Database	SQLite by default, PostgreSQL supported	No database server to install for the demo; one setting switches to PostgreSQL for anything real.
Chain access	web3.py 7.6	Submits transactions and reads events back from the chain.
Web app	Next.js 14, React 18, TypeScript, Tailwind CSS	Sixteen routes. TypeScript so the API contract is checked at build time rather than during a demo.
Testing	37 contract tests, 44 backend tests, a 56-assertion end-to-end run	The end-to-end run drives the real HTTP API against the real chain and asserts every outcome, including the attacks.

7. How the software works

Three layers, and one rule that holds the whole design together: **content stays off the chain, encrypted; only fingerprints go on it.**



The life of a paper, in eight stages:

Stage	What happens
1. Written	A setter submits a question. It is encrypted with its own AES-256-GCM key and fingerprinted before anything is stored.
2. Anchored	The fingerprint, an anonymous author marker and a timestamp are written to QuestionRegistry on the chain.
3. Guarded	Any read is checked against READ/WRITE/APPROVE grants on the server. Refusals are recorded and anchored.
4. Reviewed	A reviewer approves. The question is frozen against edits, including by its author, and the lifecycle change is recorded on the chain.
5. Assembled	A seeded optimiser selects questions against the blueprint, penalises near-duplicates (TF-IDF similarity) and spreads authorship. The same seed reproduces the same paper, so the process is auditable.
6. Sealed	The paper is encrypted, fingerprinted, and registered in PaperTimeLock with a release time.
7. Refused	Every early request is refused by the contract — including from the authority that sealed it, and including from a device whose clock has been changed.
8. Released	Once <code>block.timestamp</code> passes the release time, the contract permits release and the paper decrypts.

8. Who can do what

Role	Can	Cannot
Question Setter	Write their own questions	See another setter's question, or any paper
Reviewer	Read and approve	Edit anything, ever
Exam Authority	Assemble, seal, release	Open a sealed paper early
Auditor	Verify fingerprints and provenance	Read any question text
Super Admin	Manage accounts, inspect the audit trail	Read question content

The last row is worth saying out loud to judges: **the most privileged account in the system deliberately cannot read what the system protects**. That is unusual, and it is the direct answer to “what about a corrupt administrator?”

9. Running it from a Mac terminal

Open Terminal and run three commands. The first two are only needed once.

```
git clone https://github.com/anishc23/sih-qpleak.git
cd sih-qpleak
./scripts/start.sh
```

That single script checks your versions, installs everything, starts the local blockchain, deploys both contracts, seeds the demo data, and starts the API and the web app. First run takes a few minutes; later runs take seconds.

One thing that will bite you on a Mac

You need **Python 3.11, 3.12 or 3.13** — **not 3.14**. A required library cannot be built on 3.14 and pip fails with a long, confusing error. The script checks this first and tells you. Fix it with:

```
brew install python@3.13
```

Then open `http://localhost:3000`.

Useful afterwards:

```
backend/.venv/bin/python scripts/e2e_demo.py    # prove the whole flow
./scripts/reset_demo.sh                        # start the demo over
./scripts/stop.sh                             # stop everything
```

10. Running it from a Windows terminal

Open PowerShell and run:

```
git clone https://github.com/anishc23/sih-qpleak.git
cd sih-qpleak
.\scripts\start.ps1
```

If PowerShell refuses to run the script, allow local scripts for this session only and run it again:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Then open `http://localhost:3000`.

To prove the flow on Windows:

```
backend\.venv\Scripts\python.exe scripts\e2e_demo.py
```

Same Python rule applies: install 3.11-3.13 from python.org, not 3.14, and tick “Add Python to PATH” during installation.

Before you present, on either platform

Run `reset_demo.sh` (or `reseed` on Windows), then run the end-to-end demo **first**, and only then seal a fresh paper for the live demonstration. The end-to-end run needs a full question pool. Never reset the database without also resetting the chain — they share identifiers, and resetting only one means no paper can be sealed again.

11. The five-minute live demonstration

The full eight-beat script is in the user manual. If you only have five minutes, do these four.

Beat	Do	Say
1. The seal	Open the landing page. Do not sign in.	"That panel is reading the blockchain directly from the browser, not our server. The block number is climbing while we talk. Underneath is this laptop's own clock, struck through, because nothing consults it."
2. The attack	Sign in as the exam authority, open Time Lock, press Year 2099 , then Attempt decryption .	"I've just told the system it's the year 2099 and it believed me. The answer is still no, because the contract never asked this computer what time it is. That button calls the real API, not a mock."
3. The refusal	Sign in as a question setter and try to open another setter's question.	"This isn't a hidden button. The server checks permission on every request, and the refusal has just been written into the audit trail as evidence."
4. The tamper	As admin, open Audit Trail, confirm the chain is intact, then break one row deliberately.	"Each entry carries the fingerprint of the one before it. Editing a single line breaks every line after it — so a quiet correction is impossible."

If a judge asks whether the demo is real, open `127.0.0.1:8000/docs` and call the decrypt endpoint directly in front of them. It refuses in exactly the same way.

12. Advantages over what is used today

Question	Current systems	SecureLock
When does protection start?	When the paper exists, days before the exam	At the keystroke, weeks earlier
What is the unit of security?	The whole paper, one file, one password	Each question, its own key
Can an administrator read the content?	Usually yes	No — the highest-privileged role cannot read questions
Can the log be edited?	Yes, by whoever runs the database	Edits break the hash chain and are detected
Who decides the release moment?	A server clock, set by an administrator	A smart contract reading the blockchain's own clock
Does changing a device clock help an attacker?	Sometimes yes	No, and the attempt is recorded

Question	Current systems	SecureLock
Are refused attempts recorded?	Rarely	Always, and anchored on chain
Can a setter predict the paper?	Often, since selection is manual	No — seeded automatic assembly across many contributors
What happens if the anchor is unreachable?	Usually silent fallback	Release is refused; the failure is recorded, never faked

13. Limitations, stated plainly

Say these before a judge finds them. A team that names its own limits is far more credible than one that is caught out.

Limitation	Our position
It cannot stop a photograph	Anyone authorised to read a question can photograph the screen. No software prevents this. We reduce exposure and make access traceable.
A local test chain is not a real trust anchor	The prototype runs a local chain. It demonstrates the mechanism correctly, but production would need a permissioned or consortium chain with independent validators.
The master key is a single point of trust	Whoever holds it can decrypt the database. In production it belongs in an HSM or cloud KMS; the key-vault interface is three functions wide precisely so it can be swapped.
Question variation is rule-based, not AI	It restates the instruction and leaves the substantive clause untouched, which is exactly why the expected answer cannot drift. We label it as rule-based throughout.
Blockchain availability is a hard dependency	If the node is unreachable, release is refused. That is the correct direction to fail, but it is a real operational constraint.
block.timestamp has tolerance	Validators have some leeway. The honest claim is narrower: an end user cannot move it from their own device.
An account is not a person	We prove an authorised account acted, not which human was at the keyboard. Binding accounts to people is an organisational control, not a software one.

14. What we would build next

- **Move to a permissioned chain** with validators run by independent bodies — the board, a university, an audit authority — so no single institution controls the anchor.
- **Hardware-backed key storage** (HSM or cloud KMS) to remove the single-point-of-trust master key.
- **Two-person release**, requiring two authorised officials to co-sign before the contract will permit a release.
- **Per-centre sealed distribution**, so each examination centre receives its own sealed copy with its own release record.
- **Watermarking per reader**, so a photographed question can be traced back to the account that displayed it.

15. Questions judges are likely to ask

Question	Answer
Why blockchain? Isn't this just a database?	For most of it, yes — and we use an ordinary database for most of it. Blockchain is used in exactly two places where a database creates a trust problem: the provenance anchor, because a DBA can rewrite database history; and the release condition, because a server clock can be changed by whoever runs the server.
Are the questions stored on the blockchain?	No. Never. Only fingerprints, anonymous identifiers, timestamps and the release condition. If the entire chain were published, nobody would learn a single question from it.
What if the administrator is corrupt?	The Super Admin role cannot read question content at all, cannot release a paper early, and cannot edit the audit log without breaking the hash chain. That is the specific attacker we designed against.
What if someone changes the server time?	It changes nothing. The contract reads the blockchain's clock. We can demonstrate the equivalent attack from the client side in thirty seconds.
Is this actually running, or mocked?	Running. Every button calls the real API against a real chain. We can open the API documentation and call the endpoint directly, and show you real transaction hashes and block numbers.
How do you know it works?	37 smart-contract tests, 44 backend tests, and a 56-assertion end-to-end run that drives the real HTTP API and asserts every outcome, including the clock attack and tamper detection.
Does it need internet?	No. It runs entirely offline on one laptop. No API keys, no cloud services, no cryptocurrency.
What is the cost of running a blockchain?	Nothing here. A permissioned chain has no transaction fees; the cost is running nodes, which the participating institutions would already do. We deliberately hardcode no public network.
Can it scale to a real board exam?	The database side scales normally. The chain side writes only small fingerprints, a few per question, so throughput is modest — but honest answer: we have tested at prototype scale, not at board scale.

16. Presenter's cheat sheet

One page. Take this to the table.

THE ONE LINE

**We don't just secure the question paper.
We secure the question, from creation
to examination.**

Prompt	Answer
The problem	Papers leak weeks before the paper exists, while questions are still being written and emailed.
The idea	Secure the whole life of a question, and move the release decision somewhere no administrator can reach.
The proof	Set the clock to 2099, ask for the paper, still refused.
Where blockchain is used	Two places only: provenance anchor, release condition.
What is on the chain	Fingerprints, anonymous markers, timestamps. No question text, ever.
Roles	Setter, Reviewer, Exam Authority, Auditor, Super Admin — and the admin cannot read questions.
Numbers	37 contract tests, 44 backend tests, 56 end-to-end assertions, 16 web routes, 2 smart contracts.
Biggest limitation	Cannot stop a photograph. Say it before they ask.
Start it	Mac: <code>./scripts/start.sh</code> Windows: <code>.\scripts\start.ps1</code>
Demo accounts	<code>authority@securelock.demo</code> and four others, password <code>SecureLock#2026</code>

Final advice: lead with the problem, show the clock attack early, and volunteer your limitations. Judges reward teams who clearly understand what their system does *not* do.